RAGEN-Ward: Reproducing Multi-Turn LLM Agent Reinforcement Learning on Consumer Hardware and Probing the Algorithmic Boundary of Variance Filtering

删掉

WANG Jianyang
Student ID: 21191450
HKUST Independent Project
Repository: https://github.com/KyleWard-personal-organization/ragen_ward

Abstract

This report presents RAGEN-Ward, a consumer-hardware reproduction and extension of RAGEN for multi-turn LLM agent reinforcement learning. Under a single RTX 4070 with 8 GB VRAM, the project rewrites the RAGEN-style pipeline into modular components, adds memory and rollout-throughput optimizations, and runs a six-condition FrozenLake matrix over PPO versus GRPO and variance-filter ratios in $\{1.0, 0.5, 0.25\}$. PPO directionally reproduces the RAGEN pattern: vanilla training is unstable, moderate filtering mitigates collapse, and aggressive filtering stagnates. GRPO reverses the pattern: vanilla GRPO is the only configuration that consistently learns, reaching final reward $+0.225$, success rate 22.5%, and format compliance 80.8%. The analysis suggests that variance filtering helps PPO partly through critic-data selection, but becomes mostly sample removal for actor-only GRPO.

1 Introduction

Large language models are increasingly used as agents that act in external environments rather than as single-turn text generators. This setting is harder than ordinary preference optimization because credit assignment spans both environment turns and generated tokens, observations are only partial textual renderings of state, and rewards are often sparse.

RAGEN (Wang et al., 2025) studies this problem through State-Thinking-Actions-Reward Policy Optimization (StarPO). It combines RAGEN-style prompts, bi-level generalized advantage estimation (GAE), and StarPO-S, a rollout-filtering method that keeps prompts whose sampled trajectories have high reward variance. The original experiments assume server-grade hardware, vLLM-backed rollout, fp32 AdamW optimizer states, and multi-seed runs.

This project asks a narrower question: how much of the RAGEN training pipeline can be reproduced on a single consumer GPU, and what changes when the update algorithm is switched from PPO with a critic to critic-free GRPO? The implementation preserves the main RAGEN contracts, rewrites the system into modular components, and makes the hardware concessions explicit.

1.1 Contributions

Consumer-hardware reproduction. RAGEN-Ward rewrites the pipeline into five modules: environments, agents, RL algorithms, rollout/training, and evaluation. It adds an 8 GB VRAM optimization stack, including AdamW8bit with fp32 embedding states, gradient checkpointing, bf16 weights, micro-batching, KV-cache restoration for generation, batched rollout, and alive-only batch shrinking. These changes make 200-step 0.5B-model PPO and GRPO runs feasible on a single RTX 4070.

Algorithmic boundary of variance filtering. The project completes a 2×3 experiment matrix over algorithm $\in$ {PPO, GRPO} and variance-filter ratio $\in \{1.0, 0.5, 0.25\}$. PPO directionally reproduces the RAGEN story: vanilla training collapses, moderate filtering mitigates collapse, and aggressive filtering becomes stagnant. GRPO reverses the pattern: vanilla GRPO is the strongest configuration. This suggests that variance filtering is not algorithm-agnostic; it helps PPO partly through critic-data selection, but does not transfer directly to actor-only GRPO.

introduction章节不分subsection1.1
最后一段，先讲organization。然后说总结这两点contribution。
The paper first gives the necessary XXX.The contribution of this report is summerized as follow:
· Consumer-hardware reproduction. XXX
·Algorithmic boundary of variance filtering.XXX

1.2 Organization

The paper first gives the necessary background and system design, then presents the experiments in the conventional order: experimental setup, main results, and analysis. Supplementary curves, detailed tables, and training commands are moved to the appendix.

2 Background

2.1 LLM Agent Reinforcement Learning

In LLM agent RL, the policy π_θ is a language model. At each environment turn, the model receives a textual observation and produces a response containing both reasoning text and an action. The environment parses the action, transitions to the next state, and returns a reward. This differs from single-turn RLHF (Ouyang et al., 2022), preference optimization such as DPO (Rafailov et al., 2023), and reasoning-oriented RL such as DeepSeek-R1 (Guo et al., 2025), because the generated tokens are embedded in an external multi-turn trajectory.

The objective is to maximize expected return,

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t r_t\right], \tag{1}$$

where r_t is the turn-level environment reward. This project follows RAGEN in using sparse FrozenLake rewards, a small format penalty, KL regularization against a frozen reference policy, and bi-level credit assignment.

2.2 PPO and GRPO

PPO (Schulman et al., 2017) uses an old policy $\pi_{\theta_{\text{old}}}$, an importance ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$, and a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min(u_t, v_t)\right], \tag{2}$$

$$u_t = r_t(\theta)\hat{A}_t, \tag{3}$$

$$v_t = \tilde{r}_t(\theta)\hat{A}_t, \tag{4}$$

$$\tilde{r}_t(\theta) = \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big). \tag{5}$$

The implementation adds a value loss, entropy regularization, and KL regularization. It uses the RAGEN-aligned coefficients $c_{\text{vf}} = 1.0$, $c_{\text{ent}} = 0.001$, $\beta = 0.001$, and $\epsilon = 0.2$. PPO computes advantages with bi-level GAE: a turn-level pass estimates value at environment boundaries, and a token-level pass distributes the turn signal across response tokens.

Module	Responsibility
`envs`	Environment reset, transition, reward, and action parsing.
`agents`	Model loading, single-request generation, and batched generation.
`rl_algos`	PPO and GRPO losses, advantages, and optimizer updates.
`ragen_core`	Rollout collection, filtering, batching, and training loops.
`evaluation`	Episode-level metrics and aggregation.

Table 1: Five-module design of RAGEN-Ward.

GRPO (Shao et al., 2024) removes the critic. Given a group of trajectories sampled from the same prompt, it converts returns into group-relative z-scores:

$$\hat{A}_i = \frac{R_i - \mu_g}{\sigma_g + \varepsilon}, \qquad \mu_g = \frac{1}{G}\sum_{j=1}^{G} R_j. \tag{6}$$

The resulting scalar advantage is applied to the response tokens of that trajectory. Since positive affine transformations of all rewards in a group leave the z-scores approximately unchanged, GRPO mainly uses within-group ranking and relative separation rather than absolute reward scale.

2.3 StarPO-S Filtering

StarPO organizes multi-turn LLM agent RL around textual state, thinking text, actions, and rewards. Each turn requires a think/answer output format, and malformed outputs receive a per-turn penalty of -0.1.

StarPO-S adds variance-based rollout filtering. At each update, P prompts are sampled and K trajectories are generated per prompt. For each prompt p, the trainer computes the variance of its K trajectory returns and keeps only the top $r \cdot P$ prompts, where r is `variance_filter_ratio`. RAGEN reports that moderate filtering reduces echo-trap behavior in small-model PPO-style training. The central question here is whether this mechanism still helps after switching from PPO to GRPO.

3 Method and System Design delete

3.1 Modular Rewrite

The codebase is organized around five modules with explicit contracts.

This design makes the six-run matrix a configuration-level controlled comparison. PPO and GRPO share the same environment implementation, agent interface, rollout machinery, logging, evaluation code, and seed. They differ mainly in the RL algorithm module and in whether a critic is present.

3.2 RAGEN-Compatible Rollout

The environment builds the system prompt from an environment instruction, grid vocabulary, and action list. Each user turn appends the required output format, including `<think>...</think>` `<answer>...</answer>`, and the number of actions remaining. FrozenLake and Sokoban allow a single LLM response to contain multiple atomic actions separated by `||`; the base environment executes these actions until termination, truncation, or the action budget is exhausted.

Training and evaluation use the same batched rollout core. For each prompt, multiple independent environment instances are advanced turn by turn, and the agent generates responses for all still-active trajectories in one batch. Completed trajectories are removed from later generation batches. This preserves the per-sequence sampling distribution while improving rollout throughput in native PyTorch.

3.3 Consumer-Hardware Optimization

The implementation is designed around a single 8 GB GPU. The key memory concessions are AdamW8bit optimizer states with fp32 embedding states, bf16 model weights, gradient checkpointing, micro-batch size 1 with gradient accumulation 32, and optional reference-policy construction for KL regularization. The most important throughput fix is KV-cache restoration: after HuggingFace gradient checkpointing disables `config.use_cache`, the code restores cache use for rollout generation and explicitly disables cache only inside training forward passes.

These engineering choices are not the main experimental variables; they are the feasibility layer that makes the reproduction possible under consumer hardware. A detailed optimization table is kept in Appendix B.

#	Algorithm	Ratio	Purpose
1	PPO	1.0	Vanilla StarPO baseline
2	PPO	0.5	Moderate StarPO-S filtering
3	PPO	0.25	Aggressive StarPO-S filtering
4	GRPO	1.0	Vanilla actor-only baseline
5	GRPO	0.5	Moderate filtering
6	GRPO	0.25	Aggressive filtering

Table 2: Six-run PPO/GRPO by variance-filter-ratio matrix.

4 Experimental Setup

标题改为 Experiments

4.1 Experiment Matrix

4.1 改为Benchmarks and Baselines
内容：在什么任务上跑的，baseline是谁

The experiments form a 2×3 matrix over algorithm and variance-filter ratio.

All runs train for 200 steps with seed 42, Qwen2.5-0.5B-Instruct (Yang et al., 2024), FrozenLake-v1 (Brockman et al., 2016), sparse reward, randomized 4×4 maps, and RAGEN-aligned slippery transitions $0.8/0.1/0.1$. Each training step samples $P = 8$ prompts and $K = 16$ rollouts per prompt, giving 128 trajectories before filtering.

4.2 Baseline Alignment

4.2 改为Implementation Details
讲具体的实验设置，参数等

The core algorithmic settings follow the RAGEN 0.5B FrozenLake baseline: actor learning rate 10^{-6}, critic learning rate 10^{-5}, KL coefficient 0.001, entropy coefficient 0.001, PPO clip ratio 0.2, value-loss coefficient 1.0, $\gamma = 1.0$, $\lambda = 1.0$, turn-level discount 0.95, format penalty -0.1, and effective mini-batch size 32.

The main deviations are hardware-driven: AdamW8bit replaces fp32 AdamW, native batched PyTorch rollout replaces vLLM, and later memory-pressure runs use `--max_seq_length1536` instead of the code default 2048. The project also uses one seed rather than multi-seed averaging. These concessions limit the strength of absolute performance claims, so the analysis emphasizes large relative gaps and curve shapes. The full alignment table is in Appendix B.

4.3 Evaluation Protocol

标题改为 Evaluation Metrics

Evaluation runs every 20 training steps for 200 episodes. Environment seeds are sampled by the training process RNG rather than fixed to the same prompt set across steps, and evaluation keeps the agent's sampling configuration rather than forcing greedy decoding.

The main metrics are average environment reward, success rate, format compliance, action valid-

ity, action effectiveness, and trajectory-level reward variance. The main text focuses on reward, success, format compliance, entropy, KL penalty, gradient norm, and the number of gradient steps per update.

delete

5 Main Results and Analysis

5.1 Main Results

Table 3 gives the final state of all six runs. The central empirical result is that PPO and GRPO respond to variance filtering in opposite directions.

PPO directionally reproduces RAGEN. PPO with filter ratio 1.0 deteriorates from -0.124 at step 20 to -0.654 at step 200, reproducing the vanilla instability direction in the RAGEN small-model sparse-reward setting. Moderate filtering delays and reduces the collapse: at step 120, PPO with ratio 0.5 reaches -0.054, while vanilla PPO is already at -0.516. At the final step, the moderate filter remains 0.469 reward points better than vanilla. Aggressive filtering is stable but nearly stagnant, so its final score should not be read as a clean algorithmic win.

GRPO reverses the StarPO-S pattern. GRPO vanilla is the strongest configuration in the entire matrix, reaching reward $+0.225$, success rate 0.225, and format compliance 0.808. Filtering does not repair GRPO; both filtered GRPO variants stay near the negative-reward plateau. This is the main extension beyond reproduction: the variance filter that helps PPO does not transfer directly to actor-only GRPO.

Supplementary success, format, and multi-metric curves are moved to Appendix A. They support the same conclusion: GRPO vanilla is strongest or tied strongest across the main evaluation dimensions, while filtered GRPO is consistently weaker.

5.2 Why Filtering Helps PPO but Hurts GRPO

这句说的不完整，要自然引出为什么要分析下面的3个点

The mechanism has three parts.

Critic-data selection exists only in PPO. For PPO, high-variance prompts provide useful value-function targets. The same prompt can produce trajectories with different returns, so the critic receives informative state-return contrasts. Better critic targets improve advantage estimates and can stabilize the actor update. Variance filtering therefore acts partly as data selection for the critic.

GRPO has no critic. Its advantage is a within-group z-score. Once a group has nonzero reward variation, scaling the absolute reward spread does not proportionally scale the learning signal. Filtering high-variance groups therefore does not give GRPO the same signal amplification; it mostly removes samples.

Sample removal is more costly in GRPO. For PPO, cleaner critic data can partially offset the cost of fewer trajectories, producing a practical sweet spot. For GRPO, this benefit is much weaker. Fewer samples increase gradient variance and reduce the number of update mini-batches without a compensating critic-training gain.

The 0.25 boundary is shared. With $P = 8$, $K = 16$, and effective mini-batch size 32, the number of optimizer steps per training iteration is

$$n_{\text{grad}} = \left\lceil \frac{r \cdot P \cdot K}{32} \right\rceil . \tag{7}$$

Thus ratios 1.0, 0.5, and 0.25 produce 4, 2, and 1 optimizer steps. At ratio 0.25, the filtered buffer contains exactly one mini-batch. With one epoch, the first and only update has current log probabilities equal to old log probabilities, so the PPO-style clipping mechanism never activates. This degeneration affects both PPO and GRPO, explaining why the aggressive-filter runs look stable but stagnant.

5.3 Failure Mode Diagnostics

The vanilla PPO failure is not the same as RAGEN's classic echo trap. RAGEN echo trap is usually associated with repeated invalid modes and decreasing entropy. Here, PPO vanilla shows falling reward, zero format compliance, a sharp KL spike, and increasing entropy. This is better described as format collapse: the policy moves into malformed high-entropy output rather than a deterministic repeated phrase.

GRPO vanilla has the opposite shape. Its entropy decreases while reward, success, and format compliance increase. This is healthy mode convergence, not echo trap. The diagnostic lesson is that entropy alone is ambiguous; reward and format compliance must be interpreted jointly. Appendix A includes the supporting entropy, KL/gradient, head-to-head, and echo-proxy figures.

5.4 Quantization and Algorithm Scale

The PPO runs have pre-clip gradient norms in the hundreds or thousands, while GRPO remains

#	Run	Algorithm	Filter	Reward	Success	Format
1	PPO vanilla	PPO	1.0	−0.654	0.000	0.000
2	PPO StarPO-S moderate	PPO	0.5	−0.185	0.000	0.000
3	PPO StarPO-S aggressive	PPO	0.25	−0.060	0.005	0.050
4	GRPO vanilla	GRPO	1.0	**+0.225**	**0.225**	**0.808**
5	GRPO filtered moderate	GRPO	0.5	−0.079	0.005	0.369
6	GRPO filtered aggressive	GRPO	0.25	−0.072	0.020	0.363

Table 3: Final evaluation metrics at step 200. GRPO vanilla is the only run with sustained positive reward, non-trivial success, and high format compliance.

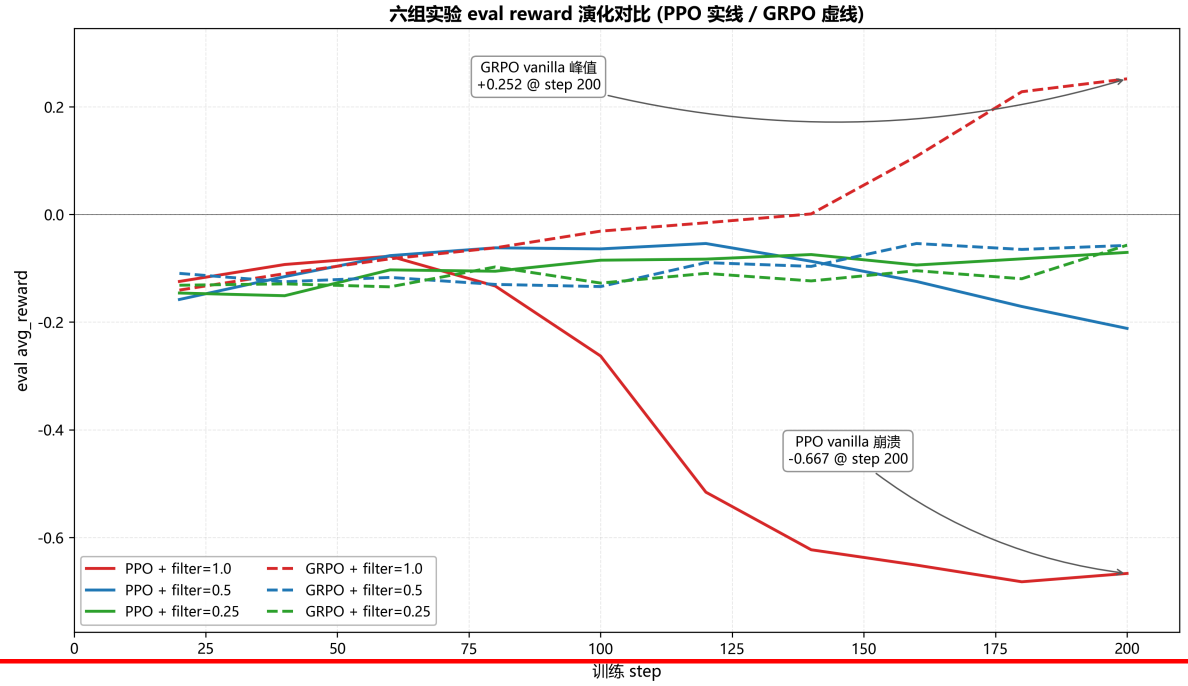

Figure 1: Evaluation reward over all six runs. PPO follows the expected filtering trade-off direction, while GRPO performs best without variance filtering.

图表里所有的中文要替换成英文，包括appendix的

around 0.05–0.16. This matches the different advantage scales: PPO's bi-level GAE can produce large token-level weights, while GRPO z-scores are naturally bounded. Since AdamW8bit introduces block-wise optimizer-state quantization noise (Dettmers et al., 2022), PPO's larger update scale and critic coupling may be more sensitive to numerical drift. GRPO provides a useful contrast: it uses the same hardware concessions but avoids the critic and uses smaller advantages, and it is the only configuration that learns robustly in this setting.

6 Limitations and Future Work

6.1 Limitations

The strongest limitation is that each configuration is run once with seed 42. RL training is seed-sensitive, so absolute reward values and exact collapse timing should not be overinterpreted. The main conclusions rely on larger relative gaps and consistent curve shapes, especially the contrast between GRPO vanilla and filtered GRPO.

The full training matrix is also limited to FrozenLake. The codebase implements other RAGEN-style environments, including Bandit, CartPole, Sokoban, and Math-Countdown, but they are not trained in this report. The mechanism claim should therefore be read as applying to the observed setting: sparse randomized FrozenLake, a 0.5B model, one seed, and consumer hardware.

Finally, the hardware concessions are not fully isolated. AdamW8bit is necessary under the 8 GB budget, and the hypothesis that it contributes to PPO instability is supported indirectly by the GRPO contrast rather than by a direct fp32 AdamW reproduction. Filtering also changes both sample selection and sample count, so equal-sample fil-

tered controls remain future work.

6.2 Future Work

The next step is a multi-seed reproduction of the six-run matrix, followed by cross-environment training on Sokoban and Math-Countdown. These experiments would test whether the PPO/GRPO filtering boundary persists beyond FrozenLake.

A second direction is to isolate the hardware and optimization effects. A larger-memory run with fp32 AdamW, or at least a less aggressive optimizer concession, would directly test the quantization-noise hypothesis. Equal-sample filter controls and a finer filter sweep, such as ratio 0.75, would separate true filtering benefits from reduced update count.

Finally, the ratio 0.25 setting should be rerun with more than one effective update opportunity, for example by increasing `ppo_epochs` or reducing the mini-batch size. This would distinguish aggressive filtering itself from the single-mini-batch degeneration observed here.

7 Conclusion Conclusion一般不分段

RAGEN-Ward reproduces a meaningful subset of RAGEN-style multi-turn LLM agent reinforcement learning on a single 8 GB consumer GPU. The modular rewrite and memory/throughput optimizations make 200-step 0.5B PPO and GRPO experiments feasible under constraints far below the original server-grade setup.

The six-run matrix gives a two-sided result. PPO directionally supports the RAGEN StarPO-S story: vanilla training is unstable, moderate filtering mitigates collapse, and aggressive filtering stagnates. GRPO exposes the boundary of that mechanism: without a critic, variance filtering mostly reduces samples and update opportunities, while group-relative z-scored advantages remove much of the reward-scale benefit. In this setting, vanilla GRPO is the best configuration.

The practical lesson is narrow but useful: for sparse multi-turn LLM agent RL under tight memory constraints, GRPO without PPO-oriented variance filtering should be tested before assuming StarPO-S transfers unchanged.

References

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. *Preprint*, arXiv:1606.01540.

Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. 8-bit optimizers via block-wise quantization. In *International Conference on Learning Representations*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *Preprint*, arXiv:2501.12948.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *Preprint*, arXiv:2402.03300.

Z. Wang, K. Wang, Q. Wang, P. Zhang, L. Li, Z. Yang, K. Yu, M. N. Nguyen, L. Liu, E. Gottlieb, M. Lam, Y. Lu, K. Cho, J. Wu, L. Fei-Fei, L. Wang, Y. Choi, and M. Li. 2025. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning. *Preprint*, arXiv:2504.20073.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, and 1 others. 2024. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

A Supplementary Results Figures

和appendix.C合并，为Additional Results
另外：apppendix的每个figure和Table，需要有citation，要么在正文里有提及，要么在appendix中要有文字说明补充了什么实验结果的数据，如图X和表X所示
单单有图标的caption是不足的

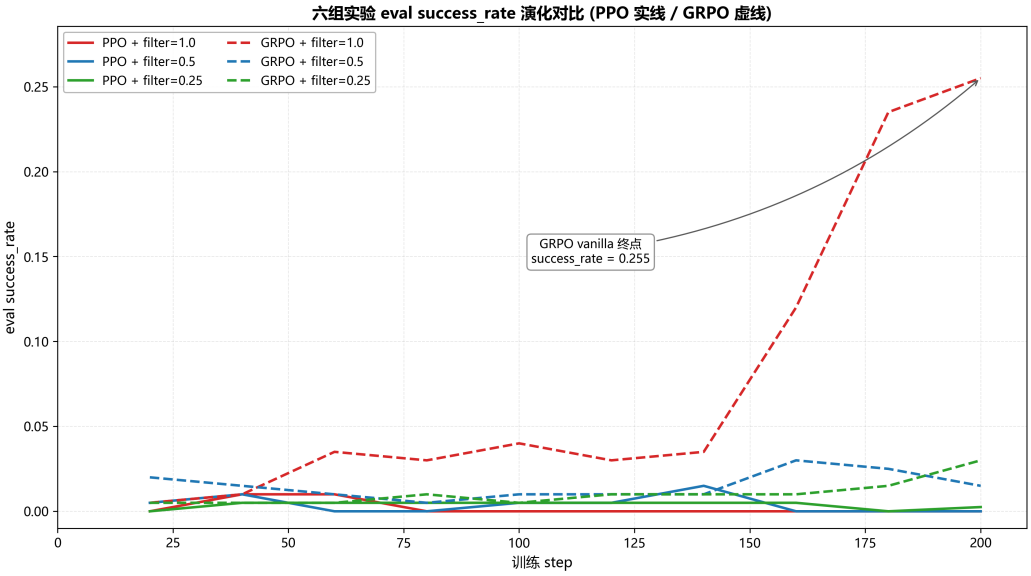

Figure 2: Evaluation success rate. GRPO vanilla is the only run with substantial task success at the end of training.

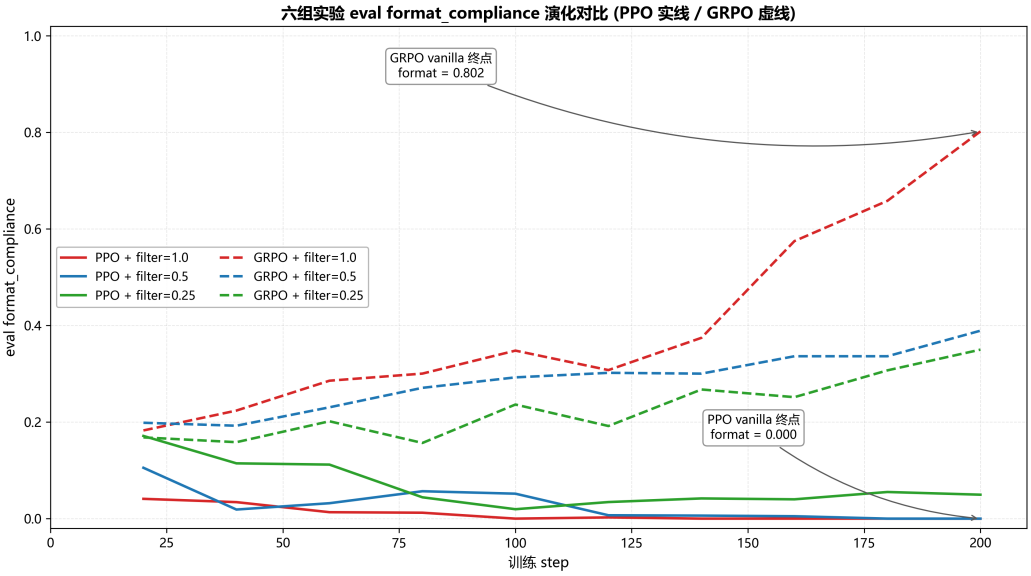

Figure 3: Format compliance. GRPO vanilla rises to approximately 0.81, while PPO variants and filtered GRPO variants remain much weaker.

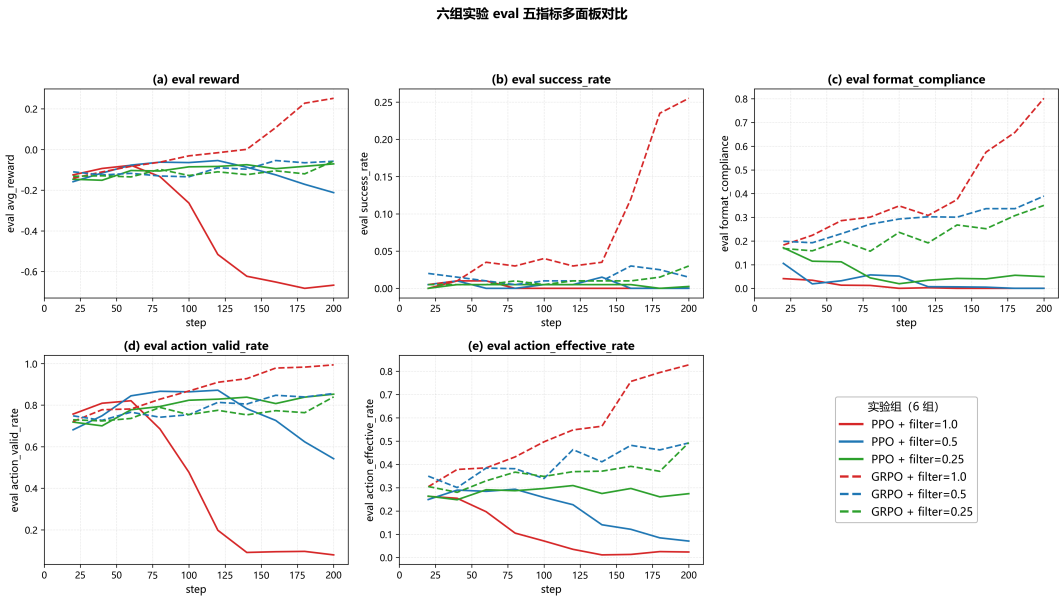

Figure 4: Five evaluation metrics. GRPO vanilla is strongest or tied strongest across the main evaluation dimensions.

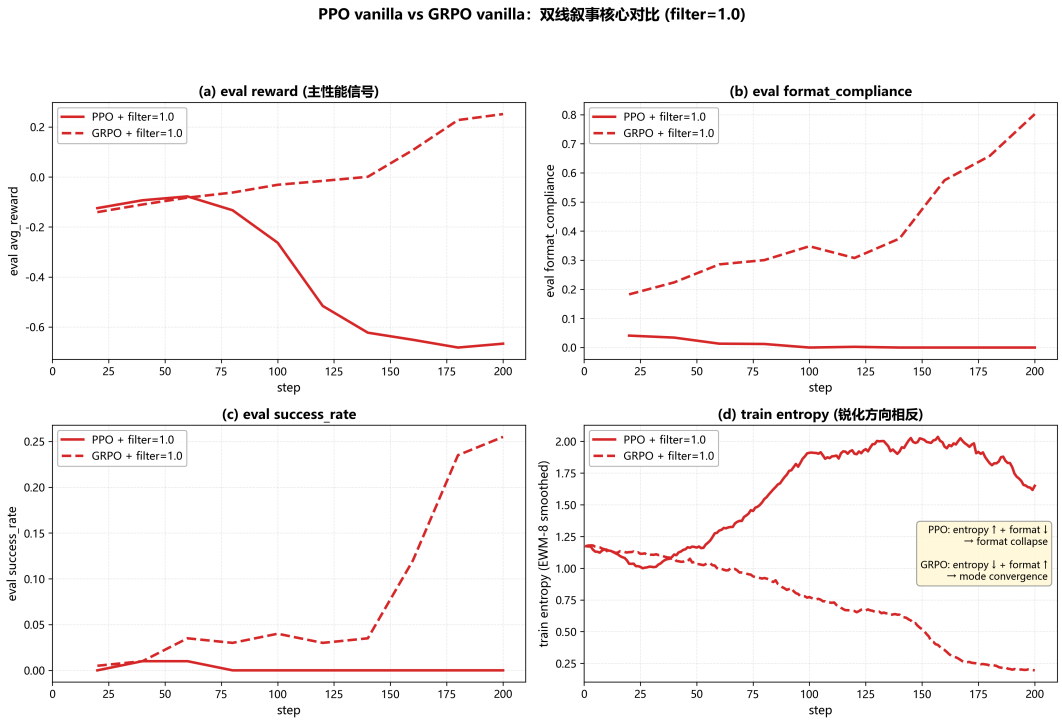

Figure 5: Head-to-head comparison of PPO vanilla and GRPO vanilla. The two runs move in opposite directions across reward, entropy, format compliance, and success rate.

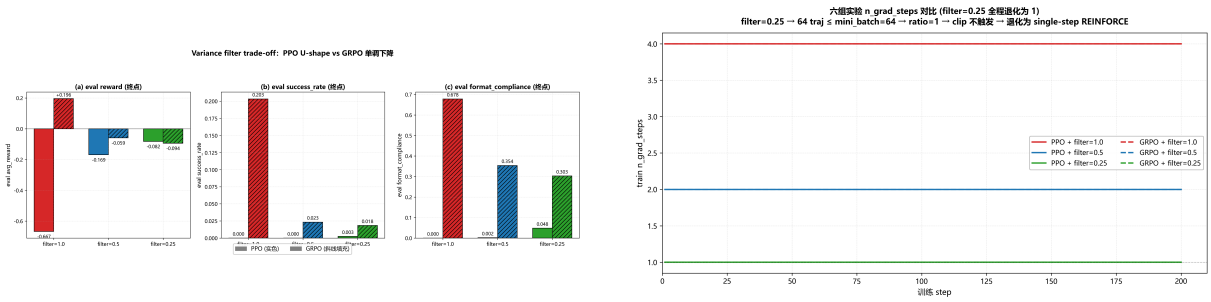

Figure 6: Left: final-reward view of the filter trade-off. Right: number of optimizer steps under each filter ratio.

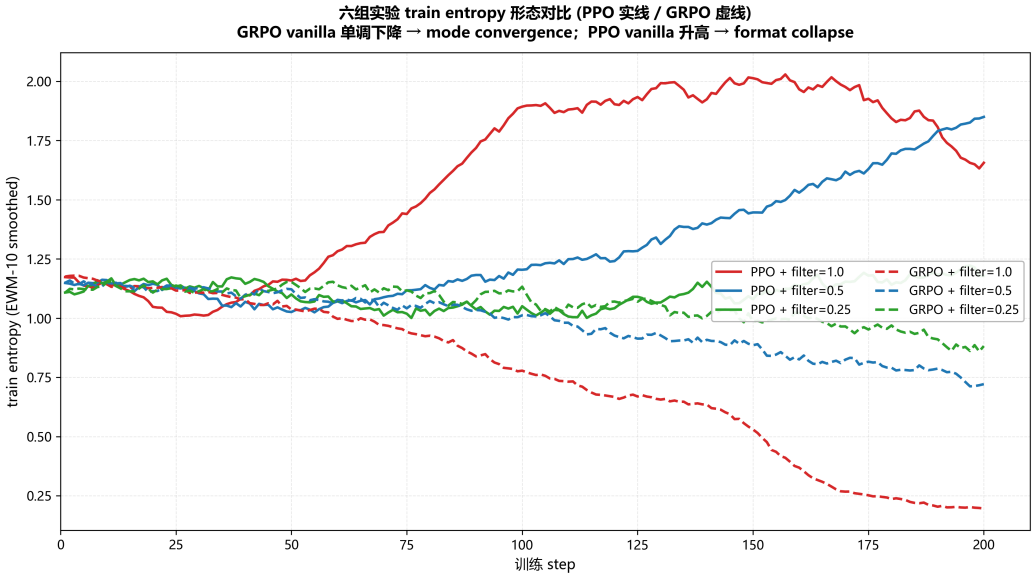

Figure 7: Training entropy. PPO vanilla moves toward higher entropy during collapse, while GRPO vanilla decreases entropy while improving reward and format compliance.

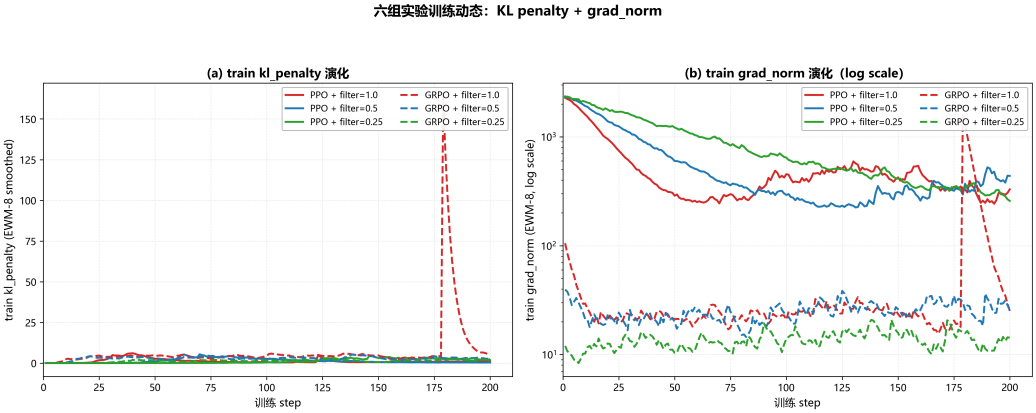

Figure 8: KL penalty and pre-clip gradient norm. PPO runs have much larger gradient norms than GRPO runs, and PPO vanilla shows a large KL spike near the collapse boundary.

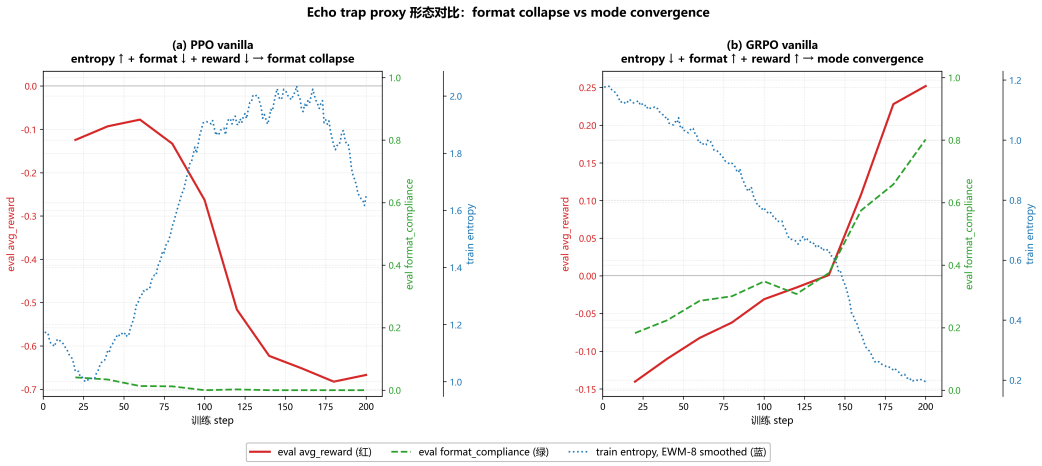

Figure 9: Joint echo-trap proxy. Entropy alone is ambiguous; reward and format compliance separate unhealthy collapse from healthy mode convergence.

B Supplementary Implementation Details

Setting	RAGEN-Ward	RAGEN baseline	Status
Model	Qwen2.5-0.5B-Instruct	Qwen2.5-0.5B-Instruct	aligned
Environment	FrozenLake-v1	FrozenLake-v1	aligned
Training steps	200	200	aligned
Prompt batch / rollouts	$8/16$	$8/16$	aligned
KL coefficient	0.001	0.001	aligned
Entropy coefficient	0.001	0.001	aligned
PPO actor / critic LR	$10^{-6}/10^{-5}$	$10^{-6}/10^{-5}$	aligned
Clip ratio	0.2	0.2	aligned
GAE settings	$\gamma = 1.0, \lambda = 1.0, \gamma_{\text{turn}} = 0.95$	same	aligned
Optimizer	AdamW8bit plus fp32 embeddings	fp32 AdamW	hardware concession
Rollout engine	Native PyTorch batched rollout	vLLM-backed rollout	software concession
Micro-batch / accumulation	$1 \times 32 = 32$	$4 \times 8 = 32$	equivalent mini-batch
Max sequence length	default 2048; some runs use 1536	3600	bounded concession
Seeds	one seed, 42	multi-seed average	major limitation
Eval episodes	200	500	higher evaluation variance

Table 4: Parameter alignment and hardware concessions relative to the RAGEN baseline.

#	Optimization	Effect and mathematical status
A	AdamW8bit	Saves optimizer-state memory, but introduces block-wise quantization noise.
B	32-bit embedding states	Keeps embedding optimizer states in fp32 to prevent NaNs.
C	Gradient checkpointing	Saves activation memory at the cost of recomputation; mathematically equivalent.
D	KV-cache restoration	Re-enables generation cache after checkpointing disables it globally; improves rollout speed.
E	Reference no-checkpointing	Disables checkpointing for the frozen reference model, whose forward passes run under no-gradient mode.
F	Micro-batch 1, accumulation 32	Preserves effective mini-batch size 32; mathematically equivalent but slower.
G	bf16 weights	Saves model-weight memory relative to fp32 and avoids fp16 range issues.
H	Max sequence length	Later runs use 1536 under memory pressure; this truncates only rare long trajectories.
I	Expandable allocator and cache clearing	Reduces effective fragmentation in Windows WDDM edge cases.

Table 5: Memory and stability optimizations used to make training feasible on an 8 GB GPU.

C Detailed Evaluation Tables

C.1 Average Reward

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	-0.124	-0.158	-0.146	-0.038	-0.105	-0.140
40	-0.093	-0.116	-0.151	-0.026	-0.103	-0.131
60	-0.078	-0.077	-0.103	-0.045	-0.078	-0.105
80	-0.133	-0.062	-0.106	+0.022	-0.108	-0.110
100	-0.263	-0.064	-0.085	-0.013	-0.111	-0.073
120	-0.516	-0.054	-0.083	+0.012	-0.064	-0.090
140	-0.623	-0.087	-0.075	+0.029	-0.046	-0.072
160	-0.652	-0.125	-0.094	+0.139	-0.026	-0.078
180	-0.683	-0.171	-0.083	+0.228	-0.075	-0.026
200	-0.654	-0.185	-0.060	+0.225	-0.079	-0.072

Table 6: Evaluation average reward over training.

C.2 Success Rate

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	0.000	0.000	0.000	0.080	0.005	0.000
40	0.000	0.000	0.000	0.085	0.005	0.000
60	0.000	0.000	0.000	0.075	0.020	0.005
80	0.000	0.000	0.000	0.140	0.005	0.005
100	0.000	0.000	0.000	0.115	0.005	0.020
120	0.000	0.005	0.005	0.130	0.025	0.010
140	0.000	0.000	0.005	0.140	0.025	0.025
160	0.000	0.000	0.005	0.215	0.040	0.020
180	0.000	0.000	0.005	0.245	0.005	0.030
200	0.000	0.000	0.005	0.225	0.005	0.020

Table 7: Evaluation success rate over training.

C.3 Format Compliance

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	0.220	0.230	0.205	0.265	0.235	0.220
40	0.180	0.260	0.220	0.305	0.290	0.265
60	0.200	0.275	0.270	0.345	0.310	0.330
80	0.155	0.305	0.305	0.450	0.320	0.355
100	0.085	0.305	0.330	0.430	0.340	0.395
120	0.030	0.285	0.345	0.495	0.350	0.425
140	0.005	0.230	0.345	0.625	0.345	0.420
160	0.005	0.150	0.290	0.685	0.335	0.380
180	0.000	0.080	0.180	0.795	0.330	0.385
200	0.000	0.000	0.050	0.808	0.369	0.363

Table 8: Evaluation format compliance over training.

C.4 Training Metric Snapshot

Metric	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
`n_grad_steps`	4	2	1	4	2	1
Entropy at step 200	~ 1.93	~ 1.82	~ 1.18	~ 0.19	~ 0.85	~ 1.16
Grad norm at step 200	~ 120	~ 80	~ 95	~ 0.16	~ 0.10	~ 0.05
KL penalty at step 200	~ 3.0	~ 1.5	~ 0.8	< 0.01	< 0.01	< 0.01
Clip behavior	active	active	zero	active	active	zero
Approx. KL	positive	positive	zero	positive	positive	zero

Table 9: Training metric snapshot used to diagnose degeneration and optimizer-scale differences.

D Training Commands

The current code default for `max_seq_length` is 2048. The commands below record the reproducible later-run form with an explicit 1536 override. The first no-filter run was originally run with the 2048 default; later runs used the 1536 override because of memory pressure after roughly 100 training iterations.

```
python scripts/train.py \\
  --algo ppo \\
  --variance_filter_ratio 1.0 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_ppo_nofilter

python scripts/train.py \\
  --algo ppo \\
  --variance_filter_ratio 0.5 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_ppo_filter05

python scripts/train.py \\
  --algo ppo \\
  --variance_filter_ratio 0.25 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_ppo_filter025

python scripts/train.py \\
  --algo grpo \\
  --variance_filter_ratio 1.0 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_grpo_nofilter

python scripts/train.py \\
  --algo grpo \\
  --variance_filter_ratio 0.5 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_grpo_filter05

python scripts/train.py \\
  --algo grpo \\
  --variance_filter_ratio 0.25 \\
  --max_seq_length 1536 \\
  --exp_name ragen_baseline_0.5B_grpo_filter025
```

The Windows environment variable used for memory-fragmentation mitigation is:

```
$env:PYTORCH_CUDA_ALLOC_CONF = "expandable_segments:True"
```